

All Pair Shortest Path Algorithms

Edited slides of
Routing Algorithms by Prof Pallab Dasgupta

Content: Distributed Algorithms by Gerald Tel : Chapter 4



Basics

When a packet is to be sent through the network, it must be forwarded using the routing tables

The routing tables must be computed when the network is initialized and must be brought up-to-date if the topology of the network changes

Basic Assumption: Undirected graphs, no negative weight cycles

Performance Issues

Correctness : The algorithm must deliver every packet to its ultimate destination

Complexity : The algorithm for the computation of the tables must use as few messages, time, and storage as possible

Efficiency : The algorithm must send packets through *good* paths

Robustness : In the case of a topological change, the algorithm updates the routing tables appropriately

Fairness : The algorithm must provide service to every user in the same degree

Good paths ...

Minimum hop: The cost of a path is the number of hops

Shortest path: Each channel has a non-negative cost – the path cost is the sum of the cost of the edges. Packets are routed along shortest paths.

Minimum delay/congestion: The bandwidth of a path is the minimum among the bandwidths of the channels on that path.

Most robust path: Given the probability of packet drops in each channel, packets are to be routed along the most reliable paths.

Destination-based Forwarding

// A packet with destination d was received or generated at node u

if $d = u$

then deliver the packet locally

else send the packet to $table_lookup_u(d)$ **//neighbour of u for destination d**

All pair Shortest Path Algorithm

To create the routing table from every node to every other node.

Use Floyd Warshall Algorithm – In every iteration add a node to the output set S where the current output set S satisfies the property that between every pair of vertices, the shortest path passing through nodes in S have been computed.

Floyd-Warshall Algorithm

begin

$S = \Phi;$

forall u, v do

if $u = v$ then $D[u, v] = 0$

else if $uv \in E$ then $D[u, v] = w_{u,v}$

else $D[u, v] = \infty;$

while $S \neq V$ do // *Loop invariant: $\forall u, v: D[u, v] = d^S(u, v)$*

begin pick w from $V \setminus S;$

forall $u \in V$ do

forall $v \in V$ do

$D[u, v] = \min\{ D[u, v], D[u, w] + D[w, v] \}$

$S = S \cup \{ w \}$

end

end

Time Complexity ?

An additional matrix stores $B[u,v]=w$ to indicate that shortest path from u to v is via w . This matrix helps reconstruct the path

Basic Assumptions of Floyd Warshall

Each cycle in the network has positive weight

Each node in the network initially knows the identities of all nodes (the set V)

Each node knows which of the nodes are its neighbours and the weights of its outgoing channels

The simple distributed algorithm

// For node u ...this happens on every node

var S_u : set of nodes;

D_u : array of weights; *// $D_u[v] = w_{u,v}$ if uv was an edge*

Nb_u : array of nodes; *// $Nb_u[v] = w$ implies that w is the first hop on the shortest path from u to v*

begin

$S_u = \Phi$;

forall $v \in V$ do

if $v = u$ then

begin $D_u[v] = 0$; $Nb_u[v] = u$ end

else if $v \in Neigh_u$ then

begin $D_u[v] = w_{u,v}$; $Nb_u[v] = v$ end

else begin $D_u[v] = \infty$; $Nb_u[v] = u$ end;

Initialisation

while $S_u \neq V$ do

begin pick w from $V \setminus S_u$; *// All nodes must pick the same w*

if $u = w$

then *broadcast the table* D_w

else *receive the table* D_w

forall $v \in V$ do

if $D_u[w] + D_w[v] < D_u[v]$ then

begin

$D_u[v] = D_u[w] + D_w[v]$;

$Nb_u[v] = Nb_u[w]$

end ;

$S_u = S_u \cup \{ w \}$

end

For node u .
Every node has to
execute this code

Assume nodes are
known and ids are known

end

Discussion

It is however not clear how the table D_w will be broadcast to all nodes. What will be the path taken by the packet to reach each node. Toueg's Algorithm gives us clarity on this.

While running the Floyd Warshall algorithm let S be the current set of vertices selected for which D and NB have been updated, Then:

- 1) For all u , $D_u[v] = d^S(u, v)$
- 2) If $d^S(u, v) < \text{inf}$, and $u \neq v$ then, $Nb_u[v]$ is the first node on the shortest path from u to v using the vertices in S .

Important property of the simple algorithm - Toueg

Let S and w be given and suppose that

(1) for all u , $D_u[w] = d^S(u, w)$ and

(2) if $d^S(u, w) < \infty$ and $u \neq w$, then $Nb_u[w]$ is the first channel of a shortest S -path to w

Then the directed graph $G_w = (V_w, E_w)$, where

$(u \in V_w \Leftrightarrow D_u[w] < \infty)$ and

$(ux \in E_w \Leftrightarrow (u \neq w \wedge Nb_u[w] = x))$

is a tree rooted towards w .

The tree is directed towards w showing the shortest path from every node u in the graph to w provided there is a path from the u to w .

Thus, u in V_w iff there is a path from u to w

and ux is an edge in E_w iff $(u \neq w \wedge Nb_u[w] = x)$

Toueg's improvement

- Toueg's observation:

- A node u for which $D_u[w] = \infty$ at the start of the w -pivot round does not change its tables during the w -pivot round.
- If $D_u[w] = \infty$ then $D_u[w] + D_w[v] < D_u[v]$ is false for every v .
- Consequently, only the nodes that belong to T_w need to receive w 's table, and the broadcast operation can be done efficiently by sending the table D_w only via the channels that belong to the tree T_w

Hence each node v forwards D_w to its neighbours if the neighbour N asserts that v is the first hop on the shortest path from N to w .

Toueg's improvement conti....

However w does not know which nodes consider w as its child in the tree T_w . Hence it sends a message to all its neighbours enquiring about the same. When it receives the information, it sends D_w to each of its parents in T_w . The parents again enquire with their neighbours to know who are its parents in the tree T_w and send D_w to them until D_w is sent to all nodes in T_w .

In each pivot round: Messages sent are E (as each node checks with its neighbours for parent information) + $N-1$ (edges on which D_w is actually sent)

Thus in total $N(E)$ messages as there are N rounds in the Floyd Warshall Algorithm

Drawback: Does not take care of channel failure
Each node needs to keep routing table information

The Chandy-Misra Algorithm (seems similar to BellmanFord)

Initiated by v_o .

When terminates, the algorithm will have the shortest path to v_o

And the neighbour on the shortest path to v_o

v_o starts by sending msgs to all its neighbours saying its distance to v_o is 0.

The neighbours on receiving this message, send messages to their neighbours saying their distance to v_o is value received on msg + weight of edge the msg is received on.

Thus, a neighbour 'a' of v_o sends msg to all its neighbours saying its distance to v_o is $0 + wt(v_o, a)$

The neighbour 'a' by send messages again if it receives messages again indicating that it has a lower distance to v_o .

Example: Another neighbour b of v_o sends a msg to the neighbour 'a' of v_o with value 1. If $wt(b, a) = 2$ then node a realises that its distance to v_o now is lesser than 5 (its 3). It then sends this new estimate to all its neighbours.

Each time the NB array is also updated where $Nba[v_o] = b$

Once shortest distances are found, a node won't need to transmit messages again and the algorithm stops

The Chandy-Misra Algorithm (seems similar to BellmanFord)

```
var  $D_u[v_0]$  : weight      init  $\infty$  ;  
     $Nb_u[v_0]$  : node      init undef ;
```

For node v_0 only:

```
begin  $D_{v_0}[v_0] = 0$  ;  
    forall  $w \in Neigh_{v_0}$  do send  $\langle \text{mydist}, v_0, 0 \rangle$  to  $w$   
end
```

Processing a $\langle \text{mydist}, v_0, d \rangle$ message from neighbor w by u :

```
{  $\langle \text{mydist}, v_0, d \rangle \in M_{wu}$  }  
begin receive  $\langle \text{mydist}, v_0, d \rangle$  from  $w$  ;  
    if  $d + \omega_{uw} < D_u[v_0]$  then  
        begin  $D_u[v_0] = d + \omega_{uw}$  ;  $Nb_u[v_0] = w$  ;  
            forall  $x \in Neigh_u$  do send  $\langle \text{mydist}, v_0, D_u[v_0] \rangle$  to  $x$   
        end  
    end
```

```
end
```


Complexity

For single source:

Bellman Ford is VE (Each node will transmit message to all its neighbours at most V times where the update by some other node reaches him)

Starting at every node we get $V(VE)$ messages in total.